

Back-end em python



Prof^a. Ma. Ana Clara

SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



Aula 25 - Padrões de Projeto e SOLID



Objetivos da Aula

- Compreender a importância dos padrões de projeto no desenvolvimento de software
- Estudar os princípios SOLID para código orientado a objetos
- Refatorar classes usando boas práticas e padrões
- Preparar os alunos para projetos maiores e mais robustos



Aula 25 - Padrões de Projeto e SOLID



O que são Padrões de Projeto?

- Soluções reutilizáveis para problemas recorrentes
- Facilitam manutenção e expansão do código
- Criam um vocabulário comum entre desenvolvedores



Aula 25 - Padrões de Projeto e SOLID

Benefícios dos Padrões de Projeto

- Melhor organização do código
- Aumenta a reutilização
- Facilita trabalho em equipe
- Ajuda a evitar erros comuns





Aula 25 - Padrões de Projeto e SOLID



Introdução ao SOLID

- Cinco princípios fundamentais para POO
- Criados para facilitar manutenção, escalabilidade e flexibilidade do software
- Muito usados em sistemas profissionais



Aula 25 - Padrões de Projeto e SOLID

Significado de SOLID

- **S**: Single Responsibility Principle (SRP)
- **O**: Open/Closed Principle (OCP)
- **L**: Liskov Substitution Principle (LSP)
- **I**: Interface Segregation Principle (ISP)
- **D**: Dependency Inversion Principle (DIP)





Aula 25 - Padrões de Projeto e SOLID



SRP - Single Responsibility Principle

- Uma classe deve ter **apenas uma razão para mudar**
- Evita classes “Deus” que fazem tudo
- Exemplo: separar lógica de cálculo e lógica de exibição



Aula 25 - Padrões de Projeto e SOLID



Exemplo SRP em Python

```
class Relatorio:
    def gerar_relatorio(self):
        pass

class RelatorioPrinter:
    def imprimir(self, relatorio):
        pass
```

Cada classe tem uma responsabilidade única



Aula 25 - Padrões de Projeto e SOLID



OCP - Open/Close Principle

- Classes devem estar abertas para extensão, mas fechadas para modificação
- Facilita adicionar novas funcionalidades sem alterar código existente



Aula 25 - Padrões de Projeto e SOLID



OCP - Open/Close Principle

```
class Desconto:
    def calcular(self, valor):
        return valor

class DescontoEstudante(Desconto):
    def calcular(self, valor):
        return valor * 0.9
```

- Novos descontos podem ser criados sem modificar a classe base



Aula 25 - Padrões de Projeto e SOLID



LSP - Liskov Substitution Principle

- Subclasses devem poder substituir suas superclasses sem quebrar o código
- Garante coerência entre herança



Aula 25 - Padrões de Projeto e SOLID



Exemplo LSP

```
class Ave:
    def voar(self):
        return "Voando"

class Pato(Ave):
    pass
```

- O pato pode substituir a classe Ave sem problemas



Aula 25 - Padrões de Projeto e SOLID



ISP - Interface Segregation Principle

- Classes não devem ser forçadas a implementar métodos que não usam
- Melhor dividir interfaces em partes menores



Aula 25 - Padrões de Projeto e SOLID



ISP - Interface Segregation Principle

```
class Imprimivel:  
    def imprimir(self):  
        pass  
  
class Escaneavel:  
    def escanear(self):  
        pass
```

- Cada interface trata apenas de sua função



Aula 25 - Padrões de Projeto e SOLID

DIP - Dependency Inversion Principle

- Módulos de alto nível não devem depender de módulos de baixo nível
- Ambos devem depender de abstrações





Aula 25 - Padrões de Projeto e SOLID



Exemplo DIP

```
class Repositorio:
    def salvar(self, dado):
        pass

class Servico:
    def __init__(self, repositorio: Repositorio):
        self.repositorio = repositorio
```

- Dependência é injetada, aumentando flexibilidade



Aula 25 - Padrões de Projeto e SOLID

Resumo de SOLID

- Código mais legível e manutenível
- Menos duplicação
- Maior escalabilidade
- Mais próximo de boas práticas do mercado





Aula 25 - Padrões de Projeto e SOLID

Atividade 1 - Refatoração com SRP

Refatorar uma classe que mistura cálculos e prints.
Separar responsabilidades em classes distintas.





Aula 25 - Padrões de Projeto e SOLID

Atividade 2 - Extensão com OCP

Criar novas classes de desconto sem modificar a base.

Exemplo: desconto de fidelidade





Aula 25 - Padrões de Projeto e SOLID

Atividade 3 - Correção de violação LSP

Corrigir hierarquia de classes onde subclasses não respeitam a superclasse.





Aula 25 - Padrões de Projeto e SOLID

Atividade 4- Interface Segregation

Dividir interfaces muito grandes em menores.

Aplicar em classes práticas..





Aula 25 - Padrões de Projeto e SOLID

Atividade 5 - Dependency Inversion

Refatorar classe de serviço que cria diretamente dependências. Usar injeção de dependência.





Aula 25 - Padrões de Projeto e SOLID

Boas práticas em Padrões de Projeto

- Não aplicar padrões sem necessidade
- Refatorar gradualmente
- Priorizar legibilidade e clareza





Aula 25 - Padrões de Projeto e SOLID



- Conhecemos os princípios SOLID
- Vimos exemplos práticos em Python
- Exercícios reforçaram boas práticas
- Próxima aula: Padrões de projeto mais usados (Factory, Singleton, Strategy)



Aula 25 - Padrões de Projeto e SOLID



Materiais de Estudo

- Livro: Padrões de Projeto – GoF
- SOLID Principles – <https://solidprinciples.com>
- Curso DevMedia: SOLID em Python
- Artigo Real Python:
<https://realpython.com/python-refactoring/>



Aula 25 - Padrões de Projeto e SOLID



Alguma dúvida ?

clara.gomes@ufpe.br



SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO